

Continuação direta do 1º semestre. A turma chega sabendo chamar a Ollama Cloud, escrever um chatbot com histórico e aplicar Zero-shot, Few-shot, CoT e Role prompting. O 2º semestre aprofunda onde o 1º parou.

📖 14 Aulas · 4 Módulos

🎯 Turmas com base do 1º sem

🔗 Python desde a Aula 01

🏁 3 CKPs com código

🕒 1h40min · 5 blocos fixos

🔗 O QUE O ALUNO TRAZ DO 1º SEMESTRE

Vocabulário consolidado: token, contexto, temperatura, alucinação, embedding
— não precisam ser reexplicados

Chatbot com histórico: lista `messages`, roles user/assistant, loop de conversa (~40 linhas)

Consciência ética: viés, LGPD, alucinações, prompt injection básico, guardrail simples

Python básico: funções, loops, f-strings, dicionários, chamada à Ollama Cloud API

Técnicas de prompt: Zero-shot, Few-shot, CoT, Role prompting, system prompt com persona

Aula 01 do 2º sem: revisão expressa de 20 min + imediatamente código LangChain rodando

 **Premissa assumida:** aula semanal numa **segunda-feira** (dia do início oficial do semestre, 03/08). Se a turma real tiver aula em outro dia da semana, nenhum destes 3 feriados cai em cima da aula e o calendário abaixo fica ainda mais folgado — **confirmar o dia da semana real de 2026.2 antes de fixar as datas com os alunos.**

AULA	DATA	SITUAÇÃO
01	03/08	Início do semestre
02	10/08	—
03	17/08	—
04	24/08	 CKP01 — entrega
05	31/08	—
 07/09 (segunda) — Independência do Brasil, aulas suspensas → aula desloca 1 semana		
06	14/09	—
07	21/09	 CKP02 — entrega
08	28/09	—
09	05/10	—
 12/10 (segunda) — N. Sra. Aparecida, aulas suspensas → aula desloca 1 semana		
10	19/10	—
11	26/10	 CKP03 — entrega (45% da nota)  antes do Kick-off GS
 02/11 (segunda) — Finados, aulas suspensas → aula desloca 1 semana · 04/11 (quarta) Kick-off Global Solutions		
12	09/11	 dentro da janela de Avaliações Semestrais/GS
13	16/11	 dentro da janela de Avaliações Semestrais/GS
14	23/11	Spec-Driven Development + encerramento · sem entrega · semana tranquila

O que mudou: as 3 datas suspensas (07/09, 12/10, 02/11) empurram cada aula seguinte 1 semana — o semestre termina em 23/11, com folga confortável até o término do período letivo (18/12). As 3 entregas de CKP (Aulas 04, 07 e 11) caem **todas antes** do Kick-off da Global Solutions (04/11) — inclusive o CKP03, que é o maior peso da nota (45%) e tem apresentação ao vivo com URL pública. As Aulas 12 e 13 (LangGraph, Módulo 4, sem CKP) caem dentro da janela institucional de Avaliações Semestrais — considere conteúdo mais leve ou revisão nessas duas semanas, já que a atenção dos alunos estará dividida com provas de outras disciplinas e a Global Solutions.



Estrutura de aula — 5 blocos · 1h40min

Mesma estrutura do 1º semestre · ritmo mais rápido nos conceitos base · profundidade maior nas aplicações

10'

ABERTURA

"O que você já sabe que vai usar hoje" + glossário dos termos novos + caso real de empresa

30'

TEORIA

Máx. 3 conceitos · definição técnica direta (sem analogia básica) · implicação prática

20'

DEMO

Notebook Demo completo ao vivo
· pode ser mais rápido que no 1º sem · aluno já tem base

30'

LAB

Notebook Aluno com lacunas — 30–50 linhas com 40–60% de lacunas · domínio do grupo

10'

SÍNTESE

3–5 bullets + conexão com a próxima aula + caso real com métrica de negócio

Diferença do 1º semestre: A teoria pode ser mais rápida porque o vocabulário base já existe. O lab começa com 30–50 linhas (vs. 8–15 no 1º sem). Os exercícios aplicam o conteúdo ao domínio de interesse do grupo — escolhido na Aula 01 e mantido até o final.

1

Módulo 1 — LangChain, Chatbots Avançados e Context Engineering

Aulas 01–04 · Python desde o primeiro dia · CKP01 ao final



Do chatbot simples ao pipeline LangChain — 4 aulas

Aulas 01–04

A turma sai do M1 sabendo usar LangChain LCEL, gerenciar contexto de forma profissional e entender por que "context rot" destrói a qualidade de um chatbot em produção.

Objetivo do módulo: Ao final da Aula 04, o aluno substitui seu chatbot "manual" do 1º semestre por um pipeline LangChain com memória gerenciada, structured output com Pydantic e consciência de context engineering. O código fica mais profissional e mais próximo do que vai encontrar no mercado.

#	TÍTULO	CONTEÚDO · DEMO · LAB
01  03/08	Revisão expressa + LangChain LCEL e ChatOllama Revisão 20min  LangChain	Revisão expressa do 1º semestre em 20 min: mapa de conceitos, o chatbot do CKP03 como ponto de partida. Por que usar LangChain: o problema de orquestrar chamadas manualmente em produção. LCEL — o operador <code> </code> como composição: <code>prompt llm parser</code>. ChatOllama conectado ao Ollama Cloud. ChatPromptTemplate com variáveis. StrOutputParser e JsonOutputParser. Demo ao vivo: reescrever o chatbot do 1º semestre em LangChain — antes: 40 linhas manuais; depois: 15 linhas LCEL. O aluno vê o mesmo comportamento com menos código. Lab (Notebook Aluno, 40% lacunas): construir a chain básica — escolher o domínio do grupo que será usado no semestre inteiro. 3 conceitos: LCEL · ChatPromptTemplate · Output Parser · Conexão 1º sem: substitui o <code>chamar_llm()</code> manual por pipeline declarativo
02  10/08	Memória conversacional — Buffer, Summary e TokenBuffer Teoria  LangChain	Por que a lista <code>messages</code> do 1º semestre não escala: custo de tokens cresce sem limite. Os 3 tipos de memória: ConversationBufferMemory (tudo na janela), ConversationSummaryMemory (resumo automático), ConversationTokenBufferMemory (limite por tokens). Quando usar cada um. ConversationChain com LangChain integrando os tipos de memória. Demo ao vivo: conversa longa (15+ turnos) com cada tipo de memória — o aluno vê o custo crescer com Buffer e estabilizar com Summary. Comparar a qualidade da resposta após 20 turnos. Lab (45% lacunas): chatbot do domínio do grupo com tipo de memória a escolher — justificar a escolha no notebook. 3 conceitos: ConversationBufferMemory · ConversationSummaryMemory · Context window cost · Conexão 1º sem: a lista messages manual agora tem gestão automática
03  17/08	Structured output e Pydantic v2 Teoria  Python	Por que forçar estrutura na saída: o LLM em produção precisa retornar JSON que outro sistema vai consumir — string livre quebra tudo. JSON Schema nativo no Ollama (<code>format="json"</code>). PydanticOutputParser: definir o schema com <code>BaseModel</code>, validar com <code>model_validate()</code>, tratar <code>ValidationError</code>. Casos de uso: extrator de entidades, gerador de relatórios, classificador de tickets.

Demo ao vivo: extrator de informações de currículos — envia texto livre, recebe JSON com nome, habilidades e experiência validados pelo Pydantic.

Lab (50% lacunas): definir o schema Pydantic para o domínio do grupo e integrar à chain existente.

3 conceitos: Structured output · BaseModel (Pydantic) · ValidationError · **Conexão 1º sem:** as f-strings de template agora têm saída validada

04

24/08

Context Engineering — de prompt engineering para context engineering

Teoria

Python

CKP01

A evolução: prompt engineering → context engineering (Anthropic, 2025). Context como recurso finito — cada token tem custo. "Context rot": como o desempenho degrada com contextos longos. O que vai no contexto: system + tools + dados + histórico. XML tagging para estruturar seções. Context caching. Meta prompting: o modelo que melhora seus próprios prompts.

Demo ao vivo: medir degradação de qualidade com contexto crescente — mesmo prompt, mesmo modelo, janelas de 500/3k/10k tokens. O gráfico de qualidade vs. tokens é a conclusão.

Lab (50% lacunas): refatorar o chatbot do grupo aplicando context engineering — medir tokens antes e depois.

3 conceitos: Context Engineering · Context rot · XML tagging · **Fonte:** Anthropic "Effective Context Engineering for AI Agents" (set/2025)

 **Check-in intermediário** — Aula 03 · QR Code no slide · 2 perguntas anônimas · resultados lidos ao vivo na Aula 04



CKP01 — "CHATBOT PROFISSIONAL" · GRUPO 3-4 · ENTREGA AULA 04 · OBRIGATÓRIO

Reescrever o chatbot do CKP03 do 1º semestre usando LangChain (LCEL + ChatOllama + memória + Pydantic). **Requisito mínimo:** ConversationChain com 1 tipo de memória + schema Pydantic validando a saída + system prompt com persona + domínio real documentado. **Diferencial:** context engineering aplicado com métricas de tokens documentadas. Estimativa: **3-4h para grupos de 4.**

2

Módulo 2 — RAG: Embeddings, ChromaDB e Avaliação

Aulas 05-07 · Pipeline RAG completo · do embedding ao RAGAS · CKP02



Do chatbot sem memória ao sistema que aprende com documentos — 3 aulas

Aulas 05–07

A turma sai do M2 com um RAG funcional sobre documentos reais, com chunking estratégico e métricas RAGAS documentadas.

Objetivo do módulo: Ao final da Aula 07, o aluno tem um pipeline RAG funcionando sobre PDFs reais com nomic-embed-text via Ollama, ChromaDB e avaliação básica com RAGAS. Sabe o que é faithfulness e answer_relevancy — e consegue medir se o seu RAG está bom ou ruim.

#	TÍTULO	CONTEÚDO · DEMO · LAB
05  31/08	Embeddings e busca semântica com ChromaDB Teoria  Python	O que é um embedding: vetor de números que representa significado — a intuição geométrica. nomic-embed-text via Ollama Cloud: 274M parâmetros, janela de 8k tokens, sem custo. Similaridade cosseno: como o sistema encontra documentos relevantes sem palavra-chave. ChromaDB: <code>add()</code>, <code>query()</code>, <code>persist()</code>, metadata filtering. OllamaEmbeddings no LangChain. Comparar busca semântica vs. keyword na mesma coleção. Demo ao vivo: indexar 20 frases sobre temas variados → buscar por "comida italiana" e ver retornar "pizza", "macarrão" e "lasanha" mesmo sem a palavra "comida". Lab (45% lacunas): criar coleção ChromaDB com textos do domínio do grupo, testar 5 queries e documentar a qualidade dos resultados. 3 conceitos: Embedding · Similaridade cosseno · ChromaDB · Stack: nomic-embed-text via Ollama Cloud (sem HuggingFace)
06  14/09	Pipeline RAG completo — load, split, embed, retrieve, generate Teoria  Python	RAG = Retrieval-Augmented Generation: o LLM não precisa saber tudo se consegue buscar. As 6 etapas: (1) carregar PDFs com PyMuPDF, (2) dividir em chunks com RecursiveCharacterTextSplitter, (3) gerar embeddings com nomic, (4) armazenar no ChromaDB, (5) recuperar os top-k mais relevantes, (6) gerar com Qwen3.6 citando as fontes. <code>create_retrieval_chain</code> no LangChain. Qual tamanho de chunk usar e por quê. Demo ao vivo: RAG sobre 3 PDFs fornecidos em sala — perguntas respondidas com citação de página e trecho. Comparar resposta do modelo sem RAG (alucina) vs. com RAG (responde com fonte). Lab (50% lacunas): RAG sobre documentos do domínio do grupo — qualquer PDF que o grupo trouxer ao Colab. 3 conceitos: Chunking · Retrieval · Grounding · Resultado: pipeline RAG funcional em ~50 linhas
07  21/09	RAG avançado — chunking estratégico, reranking e RAGAS Teoria  Python CKP02	Por que <code>chunk_size</code> importa: muito grande perde precisão, muito pequeno perde contexto. Estratégias: RecursiveCharacterTextSplitter (padrão), SemanticChunker (por significado), Parent-Document Retriever (chunk pequeno para busca, chunk grande para contexto). Reranking com cross-encoder. RAGAS: <code>faithfulness</code> (a resposta está nos docs?), <code>answer_relevancy</code> (respondeu a pergunta?), <code>context_precision</code> (recuperou os docs certos?). LLM-as-judge.

Demo ao vivo: mesmo RAG da Aula 06 testado com chunk_size 256, 512 e 1024 — medir RAGAS e mostrar qual ganhou e por quê.

Lab (55% lacunas): otimizar o RAG do grupo com 2 estratégias de chunking + calcular RAGAS básico e documentar qual configuração foi melhor.

3 conceitos: Chunking estratégico · Reranking · RAGAS (faithfulness, answer_relevancy) · **Ferramentas:** ragas · cross-encoder

 **Check-in intermediário** — Aula 07 · QR Code no slide · 2 perguntas anônimas · resultados lidos ao vivo na Aula 08



CKP02 — "DOCMIND RAG" · GRUPO 3-4 · ENTREGA AULA 07 · OBRIGATÓRIO

RAG funcional sobre base de conhecimento real escolhida pelo grupo (leis, artigos, documentação técnica, regulamentos). **Requisito mínimo:** pipeline completo (load → split → embed → store → retrieve → generate) + 2 estratégias de chunking comparadas + RAGAS faithfulness calculado para cada uma. **Diferencial:** metadata filtering + reranking + interface Gradio simples. Estimativa: **3-4h para grupos de 4.**

3

Módulo 3 — Interfaces, Agentes e Integração

Aulas 08-11 · Deploy ao vivo · Agentes ReAct · Aula 11 integradora · CKP03



Do notebook à aplicação com URL pública e ao agente autônomo — 4 aulas

A turma sai do M3 com uma aplicação publicada ao vivo e um agente que usa o RAG do grupo como ferramenta. A Aula 11 é a aula integradora — tudo junto pela primeira vez.

Aulas 08-11

Objetivo do módulo: Ao final da Aula 11, o grupo tem uma aplicação publicada com URL pública, um agente que usa ferramentas reais (RAG + DuckDuckGo + calculadora) e entende a diferença entre AI workflow e AI agent. A Aula 11 é a "vitória visível" do semestre.

#

TÍTULO

CONTEÚDO · DEMO · LAB

08



28/09

Interfaces com Gradio e Streamlit + deploy com URL pública

Teoria



Deploy

Por que interfaces importam: transformar notebook em produto. Gradio: `gr.Interface`, `gr.ChatInterface`, `gr.Blocks` — quando usar cada um. Streaming com `yield`. Streamlit: `st.chat_message()`, `st.session_state`, `st.file_uploader()`. Deploy no Colab com ngrok: URL pública temporária acessível de qualquer dispositivo. Boas práticas: nunca expor API keys na interface, timeout handling, error states.

Demo ao vivo: transformar o RAG do grupo em app Gradio em 15 linhas adicionais. Publicar via ngrok. Acessar pelo celular ao vivo em sala.

Lab (45% lacunas): cada aluno publica sua própria versão ao vivo. URL compartilhada no grupo da turma.

3 conceitos: Gradio ChatInterface · ngrok deploy · Streaming · **Resultado:** primeira URL pública de uma aplicação com IA

09



05/10

Agentes de IA — ReAct, tools e function calling

Teoria



Agentes

O que diferencia um agente de um chatbot: o loop percepção → raciocínio → ação → observação. AI Workflow vs. AI Agent — quando usar cada um. ReAct pattern: Reasoning + Acting. Tool schema: `name`, `description` (onde 80% do trabalho está), `parameters`. Function calling nativo no Qwen3.6. `@tool` decorator. AgentExecutor: `max_iterations`, `verbose=True`. Tools prontas: DuckDuckGoSearchRun, PythonREPLTool, calculadora.

Demo ao vivo: agente com 3 tools (busca + calculadora + Python REPL) resolvendo um problema multi-step. Cada tool call visível no log com `verbose=True`.

Lab (50% lacunas): agente com 3 tools via Ollama Cloud — o aluno escreve os schemas das tools e vê o agente decidir qual usar.

3 conceitos: Loop agêntico · Tool schema · AgentExecutor · **Conexão segurança:** guardrail contra prompt injection indireta via tool

10



19/10

Context Engineering para agentes — curadoria em loop agêntico

Teoria



Python

Context em agentes ≠ context em chat: o loop gera dados continuamente. Tipos de memória agêntica: in-context (scratchpad), external (ChromaDB), episódica (histórico de interações passadas). Curadoria dinâmica: o que incluir e descartar a cada iteração. "Context rot" em loops longos — medir degradação em 5/10/20 turnos ao vivo. Princípio da ação mínima. Model Context Protocol (MCP): overview do padrão para conectar agentes a ferramentas externas.

Demo ao vivo: instrumentar o agente da Aula 09 para medir qualidade da resposta a cada turno. O gráfico de qualidade vs. turnos mostra o context rot acontecendo.

Lab (55% lacunas): implementar estratégia de compressão do scratchpad e medir a diferença.

3 conceitos: Scratchpad agêntico · Context rot em loop · MCP overview · **Fonte:** Anthropic Context Engineering for AI Agents (set/2025)

11

🌟 Aula integradora — Agente com RAG como tool + interface ao vivo

Aula 100% de laboratório — nenhum conceito novo. Integrar tudo do semestre: `create_retriever_tool()` — encapsular o ChromaDB do grupo como tool do agente. Multi-source: RAG local + DuckDuckGo + Python REPL. A

`description` do tool é o que determina quando o modelo usa cada uma — dedicar tempo a escrever bem. Gradio ChatInterface com streaming em tempo real. Deploy Colab + ngrok. Guardrails de input integrados.

Demo ao vivo (25 min): professor constrói o sistema do zero ao vivo — do `create_retriever_tool()` ao deploy com URL publicada.

Lab (25 min, 60% lacunas): cada grupo publica sua versão ao vivo. URL compartilhada com a turma. Demo cruzada: cada grupo testa a app do grupo vizinho.

Sem conceito novo · Integração de M1+M2+M3 · **Resultado:** agente + RAG + interface ao vivo com URL pública — demonstrável para qualquer pessoa



Check-in intermediário — Aula 11 · QR Code no slide · 2 perguntas anônimas · resultados lidos ao vivo na Aula 12



CKP03 — "AGENTE INTELIGENTE" · GRUPO 3-4 · ENTREGA AULA 11 · OBRIGATÓRIO

Agente com pelo menos 3 tools, sendo 1 obrigatoriamente o RAG do grupo (via `create_retriever_tool()`). **Requisito mínimo:** 3 tools funcionando + interface Gradio com URL pública + 3 exemplos de interação documentados + tool descriptions bem escritas. **Diferencial:** guardrail de input + compressão de contexto + mais de 3 tools. Estimativa: **3-4h para grupos de 4**.

4

Módulo 4 — LangGraph e Spec-Driven Development

Aulas 12-14 · Avançado · LangGraph completo na Aula 13 · Aula 14 fecha com a fronteira mais atual do código assistido por IA



Grafos de estado, agentes avançados e Spec-Driven Development — 3 aulas

LangGraph para quando o AgentExecutor não é suficiente. A Aula 14 fecha o semestre com SDD — a especificação estruturada como fonte de verdade para agentes de código.

Aulas 12-14

Objetivo do módulo: Ao final da Aula 13, o aluno entende quando LangGraph é necessário (branches, loops condicionais, checkpointing, human-in-the-loop) e consegue construir um agente pesquisador com loop condicional. A Aula 12 prepara o terreno com Router Chains e o conceito de grafo. A Aula 14 é encerramento livre.

#	TÍTULO	CONTEÚDO · DEMO · LAB
12  09/11	Router chains e o conceito de grafo de estado Teoria  LangChain	Por que chains lineares não são suficientes para sistemas complexos: quando o fluxo precisa de branches e loops. Router Chain: decidir qual chain usar com base no input. Motivação para LangGraph: AgentExecutor é para loops simples; LangGraph é para quando o fluxo tem branches, paralelismo e checkpointing. Introdução ao conceito de StateGraph sem código ainda — apenas o modelo mental. Demo ao vivo: Router Chain que classifica a intenção do usuário e roteia para 3 handlers diferentes (RAG, cálculo, conversa). O aluno vê o modelo decidindo qual rota seguir. Lab (50% lacunas): router chain com 3 rotas para o domínio do grupo — o aluno escreve o classificador de intenção e os handlers. 3 conceitos: Router Chain · Grafo de estado (conceitual) · Classificador de intenção · Preparação: o modelo mental de grafo que será implementado na Aula 13
13  16/11	LangGraph — StateGraph, nodes, edges, conditional routing e HITL Teoria  LangGraph	StateGraph com TypedDict: o estado compartilhado entre todos os nodes. <code>add_node()</code> e <code>add_edge()</code> : nós e arestas. <code>add_conditional_edges()</code> : o ponto onde o grafo decide o próximo passo. MemorySaver checkpointing: salvar e retomar estado entre chamadas. Human-in-the-loop com <code>interrupt_before</code> : pausar para aprovação humana antes de ação irreversível. <code>draw_mermaid()</code> para visualizar o grafo. Devstral-Small como modelo para coding agents. Demo ao vivo: agente pesquisador com loop condicional — node de busca → node de avaliação → conditional edge (buscar mais ou responder) → node de resposta. Máximo 3 iterações. Visualizar o grafo com <code>draw_mermaid()</code> . Lab (60% lacunas): construir o mesmo agente pesquisador para o domínio do grupo. O grafo deve ter pelo menos 3 nodes e 1 conditional edge. 3 conceitos: StateGraph · Conditional routing · Human-in-the-loop · Ferramentas: langgraph · MemorySaver · draw_mermaid()
14  23/11	Spec-Driven Development e encerramento Teoria Sem entrega Sem notebook	Aula conceitual e demonstrativa — sem entrega avaliada. O que é Spec-Driven Development (SDD): a especificação estruturada, não o prompt solto, como fonte de verdade para agentes de código. SDD vs. vibe coding vs. prompt engineering direto — quando cada abordagem se aplica. O fluxo spec → plano → tasks → implementação pelo agente → revisão humana. Ferramentas: GitHub Spec Kit, Claude Code, Kiro. Conexão direta com o que a turma já pratica: spec é context engineering aplicado a código — os próprios arquivos SOUL.md/CLAUDE.md usados para gerar o material do curso são um exemplo real. Demo ao vivo: escrever uma spec curta (requisitos + critérios de aceite) e usar um agente de código gratuito para implementá-la, revisando o diff gerado contra a spec. Fecha com retrospectiva do semestre completo: do primeiro prompt conceitual ao agente com LangGraph, e os CKPs como portfólio para o mercado.



Resumo executivo — 2º semestre

14 aulas · 4 módulos · 3 CKPs com código · Continuação direta do 1º semestre

MÓDULO	FOCO	AULAS	STACK NOVO	ENTREGA
M1 — LangChain	LCEL, memória, Pydantic, Context Engineering	01-04	langchain · pydantic	CKP01 · com código
M2 — RAG	Embeddings, ChromaDB, chunking, RAGAS	05-07	chromadb · ragas	CKP02 · com código
M3 — Agentes	Gradio, ReAct, tools, integradora	08-11	gradio · langchain agents	CKP03 · com código
M4 — LangGraph	Router, StateGraph, conditional routing, HITL, Spec-Driven Development	12-14	langgraph · spec kit	Sem CKP

A progressão entre os dois semestres como um arco completo:

1º sem: o que é IA → como funciona → primeiro prompt → técnicas → chatbot com ~40 linhas.
2º sem: o chatbot vira pipeline LangChain → o pipeline vira RAG → o RAG vira tool de um agente → o agente vira grafo com LangGraph.
O aluno que completa os dois semestres consegue construir um sistema de IA completo, publicar com URL pública e explicar cada decisão de arquitetura.

Disciplina: Prompt Engineering and Artificial Intelligence · FIAP · Ciência da Computação · 2º Semestre 2026

Prof. Jorge Luiz Gomes · profjorge.gomes@fiap.com.br

Continuação do 1º Semestre · LangChain · LangGraph · ChromaDB · RAGAS · Gradio · Ollama Cloud · Oliveira (2007)